

Issued by BSE Ltd. for STAR MF V2 System Users

BSE StARMF V2 - Step by Step Encryption Guide

STEP 1: INITIAL SETUP

What You Need to Do:

1. Generate RSA keypair (minimum 2048 bit). You can use tools/libraries to generate keypair, for e.g. https://techdocs.akamai.com/iot-token-access-control/docs/generate-rsa-keys provides that, refer for understanding but recommendation is to generate using libraries in your code base.

2. Share **only your public key** with BSE API team during onboarding for configuration.

3. Share your public IP addresses (if not earlier) for API requests.

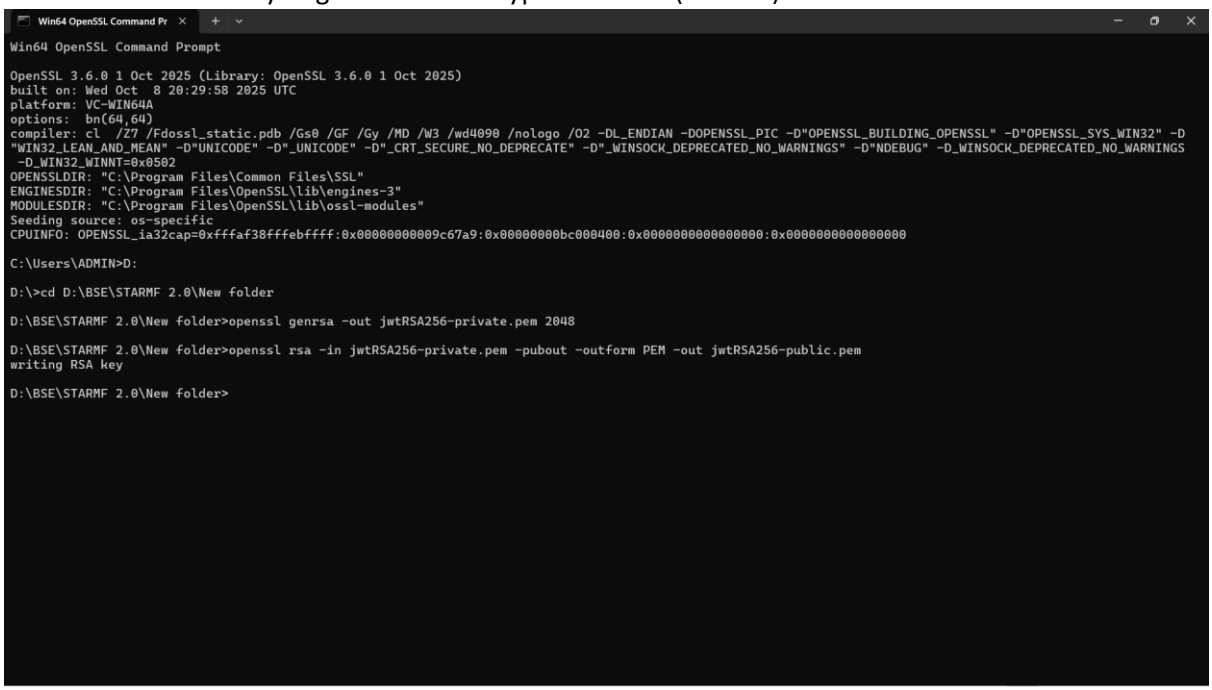
What BSE Will Give You:

1. BSE's public key

2. Your X-API-Org-ID header's fingerprint value i.e. checksum for sending requests to API.

Note:

- **RSA Keys (Member) Mandatory Specifications**: - RSA 2048-bit or higher - PEM format (base64), PKCS#8 preferred - UTF-8 encoding - No line breaks inside base64 body except standard PEM wrapping
- Public keys must include PEM boundaries. **Example**:
  -----BEGIN PUBLIC KEY-----
  MII...
  -----END PUBLIC KEY-----
- Do not send private key to BSE, do not log private keys, do not use same private key for UAT and PROD
- After installing OpenSSL, you can generate a 2048-bit RSA keypair using the sample OpenSSL commands provided in below screenshot. This is only an example, members may use any standard tool or library to generate RSA keypairs in PEM (PKCS#8) format.



STEP 2: INSTALLING LIBRARIES FOR YOUR CODEBASE

You Need:

- JOSE library for your programming language (https://jose.readthedocs.io/en/latest/)
- RSA encryption support
- Latest libraries are recommended. Below libraries can be used in your codebase:

1. **Python**: jwcrypto, cryptography
2. **Golang:** github.com/lestrrat-go/jwx, golang.org/x/crypto, crypto (standard library)
3. **Node.js / TypeScript:** jose, node-jose, jolse
4. **.NET (C#)**: Jose.JWT, System.Security.Cryptography, Microsoft.IdentityModel.Tokens
5. **Java**: Nimbus JOSE + JWT, BouncyCastle

STEP 3: ENCRYPTION (Before Sending Request):

Input: Your JSON request data:

For example:

```
{
  "data": {
    "ifsc_code": "YESB0000031",
    "account_number": "003199000002381",
    "pan": "ABCDE1223G",
    "name": "John Doe"
  }
}
```

Process in your codebase:

1. Create JOSE Encrypter:
   - Session key algorithm: AES_256_GCM
   - Recipient algorithm: RSA_OAEP_256
   - Recipient public key: BSE's public key
2. Encrypt:
   - jweObj = joseEncrypter.Encrypt(your_json_payload)
   - jweCompact = jweObj.CompactSerialize()
3. Sign:
   - joseSigner = jose.Signer(Algorithm: RS256, Key: your_private_key)
   - jwsObj = joseSigner.Sign(jweCompact)

o encPayload = jwsObj.CompactSerialize()

Output: Encrypted string to send in API request body

Note: BSE provides a sample Python script implementation at https://starmfv2wrapper.bseindia.com/documentation.
This script serves as a reference for understanding the encryption and signing flow.

STEP 4: SENDING REQUEST:

Headers to configure:

1. Content-Type: application/jose
2. X-API-Org-ID: member/<your_org_code>:<your_fingerprint>
3. Authorization: Bearer <access_token>
4. **Body**: The encrypted string from Step 3. If postman is used then the body must be sent in raw and text.

Notes:

1. Login APIs do not require an Authorization header. Every other API in StAR MF V2 requires a valid Authorization: Bearer <access_token> header.
2. BSE will share your fingerprint (checksum) and BSE's public key after you submit your public key and BSE completes the configuration.
3. <your_org_code> is the member code allocated to your organization by BSE.
   Example: 0123
4. Example of the X-API-Org-ID header format:
   X-API-Org-ID: member/0123:ab8492f13c982a1d
   Where: 0123 = your BSE member code, ab8492f13c982a1d = fingerprint shared by BSE

STEP 5: DECRYPTION (After Receiving Response)

Input: Encrypted response from BSE

Process:

1. Parse JWS:

o jwsObj = jose.ParsedSign(encrypted_response)

2. Verify Signature:

o jweBytes = jwsObj.Verify(bse_public_key)

3. Parse & Decrypt:

o jweObj = jose.ParseEncrypted(jweBytes)

o decrypted_json = jweObj.Decrypt(your_private_key)

Output: Your original JSON response data

EXAMPLE FLOW:

Your JSON → Encrypt with BSE key → Sign with your key → Send

↓

Encrypted Response ← Verify BSE signature ← Decrypt with your key ← Receive

WHAT YOU IMPLEMENT

1. ctEncrypt() function - for outgoing requests
2. ctDecrypt() function - for incoming responses
3. Header generation - for API calls
4. Key management - load your private key and BSE public key